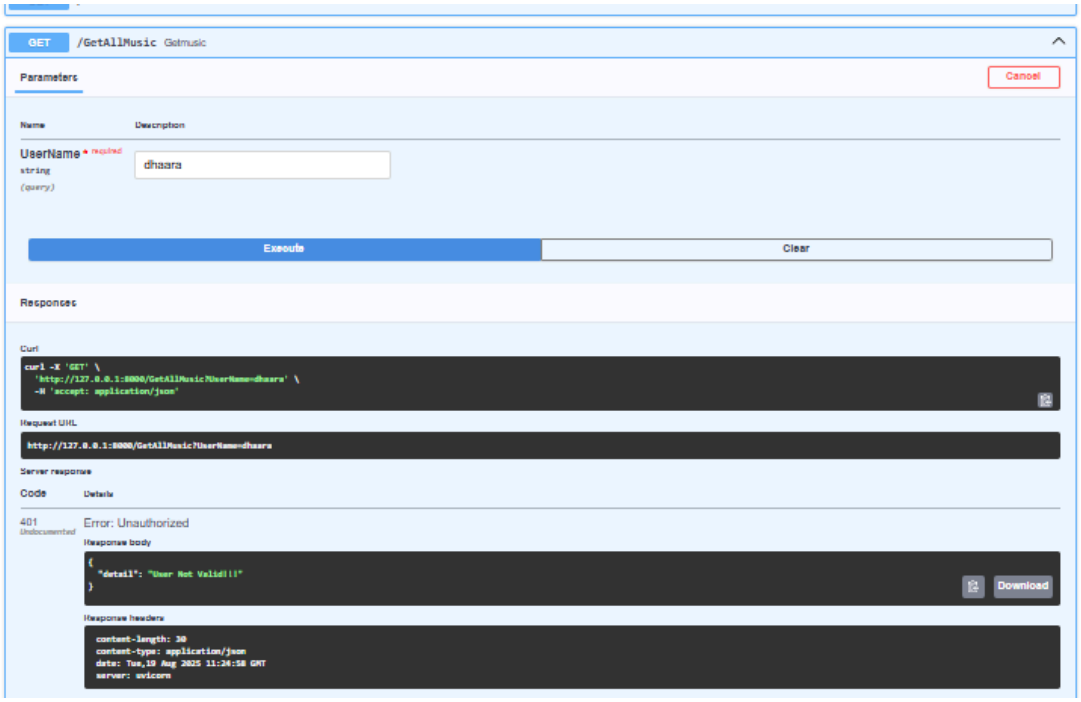
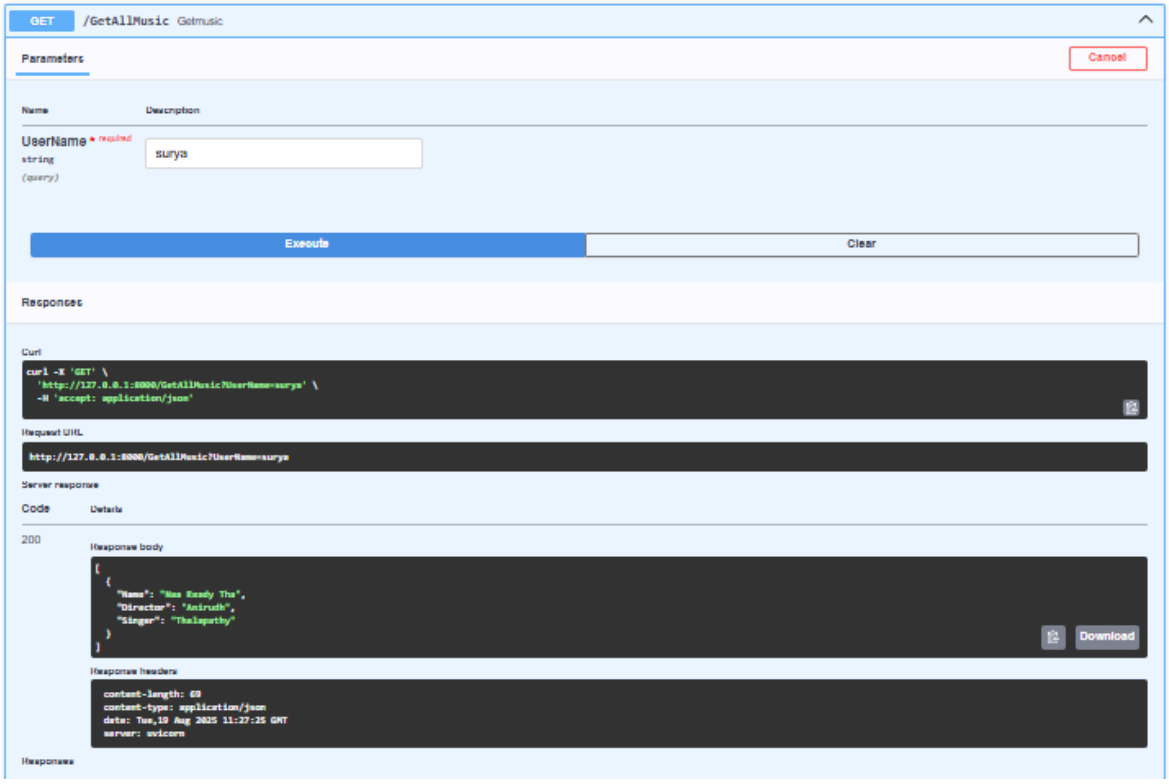
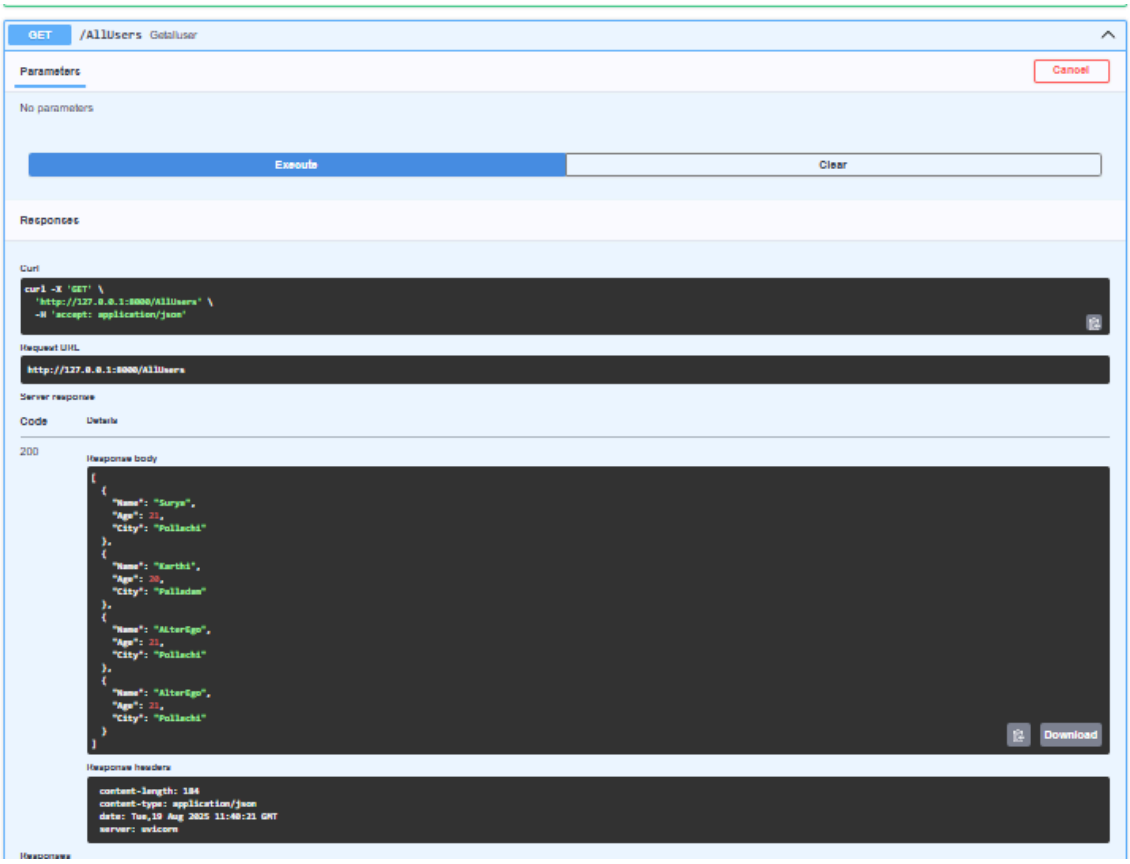
Surya Prakash M
19/8/25





Surya Prakash M
19/8/25



SqlAlchemy File:

```
from fastapi import Depends, HTTPException
from pydantic import BaseModel
from sqlalchemy import create_engine, func
from sqlalchemy.dialects.sqlite import *
from sqlalchemy.orm import sessionmaker,declarative_base,Session
url = "sqlite:///./Training.db"
engine = create_engine(url,connect_args={"check_same_thread":False})
session = sessionmaker(autocommit=False,autoflush=False,bind=engine)

Base = declarative_base()

from sqlalchemy import Column,Integer,String
class Musics(Base):
    __tablename__ = "Music"
    ID = Column(Integer,primary_key=True,index = True)
    Name = Column(String(50),nullable = False)
```

```
    Singer = Column(String(50),nullable = False)
    Director = Column(String(50),nullable = False)


Base.metadata.create_all(bind=engine)

class Music(BaseModel):
    Name: str
    Director: str
    Singer:str
    class Config:
        orm_mode = True
        from_attributes=True

class Users(Base):
    __tablename__ = "Users"
    ID = Column(Integer,primary_key=True,index = True)
    Name = Column(String(50),nullable = False)
    City = Column(String(50),nullable = False)
    Age = Column(Integer,nullable = False)

Base.metadata.create_all(bind=engine)

class User(BaseModel):
    Name: str
    Age: int
    City: str
    class Config:
        orm_mode = True
        from_attributes=True


def get_db():
    db = session()
    try:
        yield db
    finally:
        db.close()
```

```
def add_music(name,director,singer,db: Session,UserName : str):
    user = GetUserByName(db,UserName)
    if user:
        st = Musics(Name = name ,Director = director,Singer = singer)
        db.add(st)
        db.commit()
        db.refresh(st)
    else:
        raise HTTPException(status_code=401, detail="User Not Valid!!!")


def GetAllMusic(db:Session,UserName:str):
    user = GetUserByName(db,UserName)
    if user:
        MusicList =  db.query(Musics).all()
        Music_schemas = [Music.model_validate(stud) for stud in MusicList]
        return Music_schemas
    else:
        raise HTTPException(status_code=401, detail="User Not Valid!!!")

def GetMusicByName(db:Session,MusicName:str,UserName:str):
    user = GetUserByName(db,UserName)
    if user:
        music =
db.query(Musics).filter(func.lower(Musics.Name).contains(MusicName.lower()
)).first()
        if music:
            return Music.model_validate(music)
        return None
    else:
        raise HTTPException(status_code=401, detail="User Not Valid!!!")

def add_user(name,age,city ,db: Session ):
    st = Users(Name = name ,Age = age,City = city)
    db.add(st)
    db.commit()
    db.refresh(st)

def GetUserByName(db:Session,name:str):
```

```
    student =
db.query(Users).filter(func.lower(Users.Name).contains(name.lower())).firs
t()
    if student:
        return User.model_validate(student)
    return None

def GetAllUser(db:Session):
    usrList =  db.query(Users).all()
    usr_schemas = [User.model_validate(usr) for usr in usrList]
    return usr_schemas
```

Fast:

```
from fastapi import FastAPI, status,Depends,Response
import uvicorn
import MusicSql as sql
from sqlalchemy.orm import Session

app = FastAPI()

@app.post("/AddMusic")
async def Post_student(UserName:str,music: sql.Music, db: Session =
Depends(sql.get_db)):
    sql.add_music(music.Name, music.Director, music.Singer, db,UserName)
    return {"message": "Music added successfully"}

@app.post("/AddUsers")
async def AddUser(user: sql.User,db : Session = Depends(sql.get_db)):
    sql.add_user(user.Name,user.Age,user.City,db)
    return {"message" :"User Added Successfully!!"}

@app.get("/AllUsers")
async def GetAllUser(db:Session = Depends(sql.get_db)):
    return sql.GetAllUser(db)



@app.get("/GetAllMusic")
async def GetMusic(UserName:str,db:Session = Depends(sql.get_db)):
    return sql.GetAllMusic(db,UserName)
```

```python
@app.get("/GetMusicByName")
async def GetMusicByName(MusicName:str,UserName:str,response:
Response,db:Session = Depends(sql.get_db)):
    music =  sql.GetMusicByName(db,MusicName,UserName)
    if music:
        return music
    response.status_code = status.HTTP_404_NOT_FOUND
    return { "Message": "Not Found"}

if __name__ == "__main__":
    uvicorn.run("MusicFast:app",host="127.0.0.1",port=8001)
```